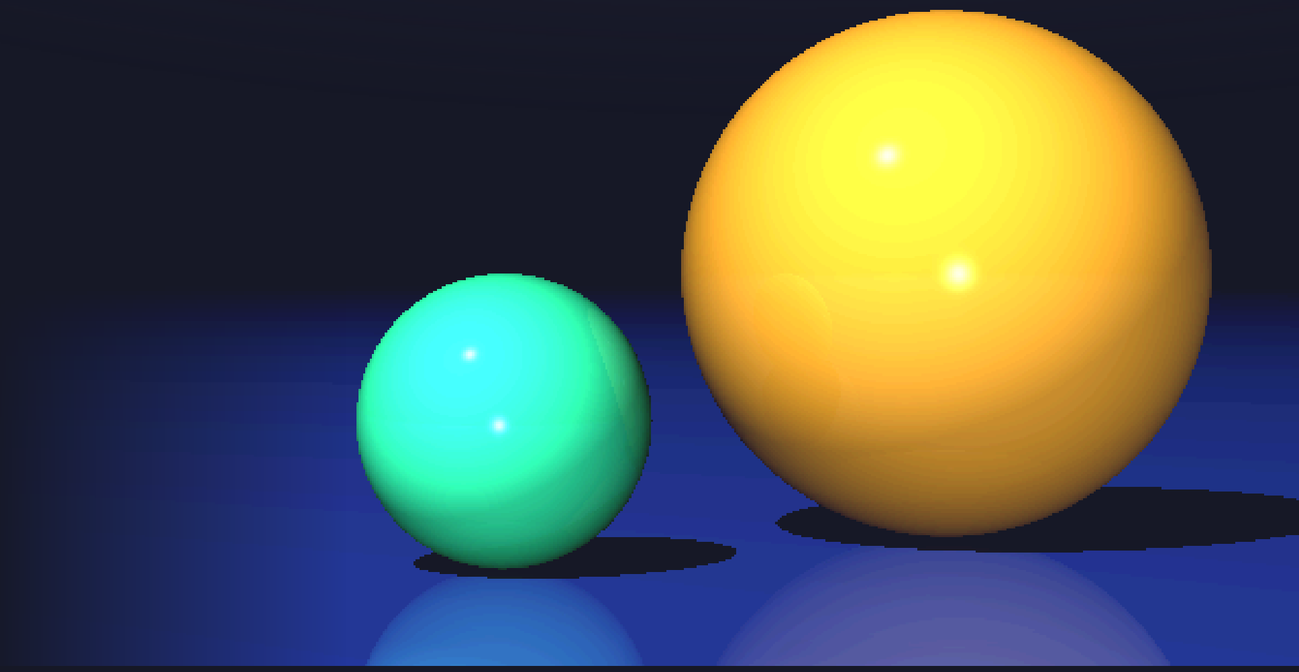
CSC317 · FALL 2026

Test 1 Review

Raster images, ray casting and ray tracing — recaps and fourteen worked practice questions.



Lectures 1–3 · Fundamentals of Computer Graphics, Ch. 3–4

The big picture

01	Raster images	A picture is a grid of coloured dots. What can we do with it?	Q1–Q7
02	Ray casting	Shoot a line from the eye through each pixel. What does it hit?	Q8–Q13
03	Ray tracing	It hit something. How bright does it look?	Q14
04	Global illumination	Real light bounces around. What are we missing?	Recap

What the test looks like

TOPIC	TYPICAL ASK	MARKS
Raster images	Pixel algorithm in pseudocode — threshold, region fill	6
Ray–object intersection	Build a new shape from given sphere / plane routines	10
Illumination	True or false, with the reason	10
Bounding volumes	Algorithm, then compare volumes	6

FORMAT 60 minutes, 32 marks, pseudocode is fine. No cheat sheet — the equation you need is given in the question. What's tested is using it and deriving from it.

01

Raster images



Images and raster devices

WHAT AN IMAGE IS

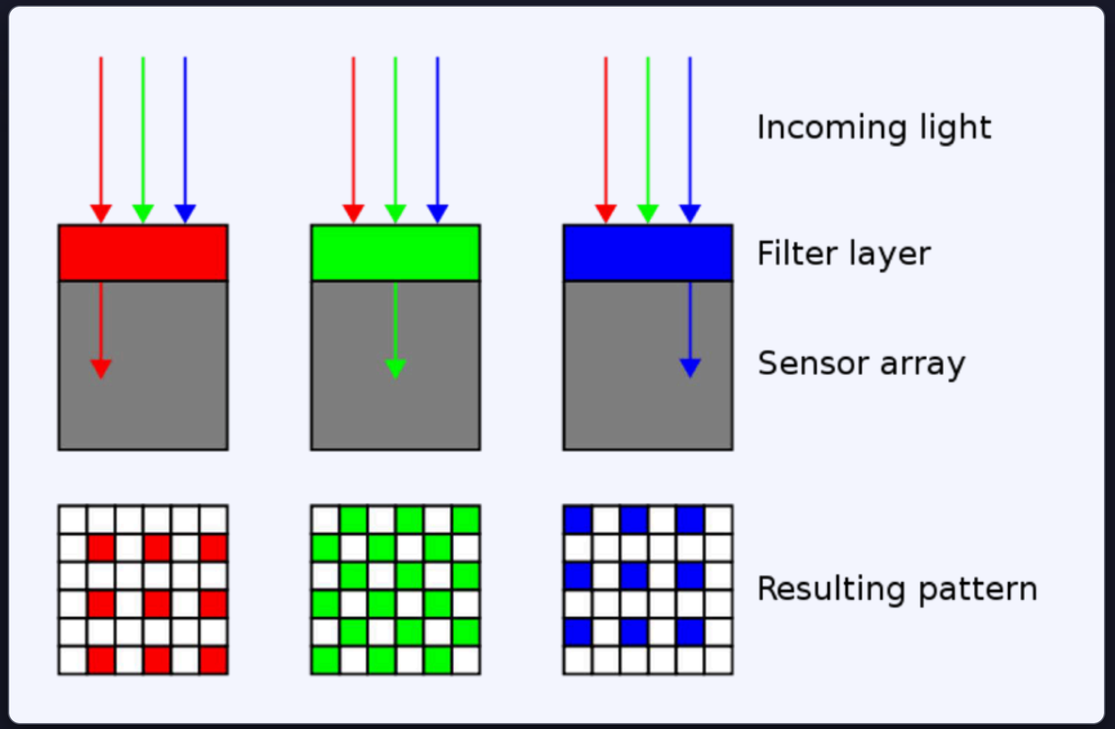
- A grid of pixels. Each pixel is the colour at one point.
- A pixel is a sample at a point, not a little square tile.

DEVICES

- Capture: cameras and scanners. Show: LED makes light, LCD filters a backlight, ink soaks light up.
- Cyan ink absorbs red, magenta absorbs green, yellow absorbs blue.

BAYER FILTER AND DEMOSAICING

- The sensor records one colour per pixel, in a checker pattern — twice as much green, because eyes care most about green.
- Demosaicing fills in the two missing colours by averaging the neighbours.



Bayer filter: one colour per pixel

RGB → HSV

$$M = \max(R, G, B) \qquad m = \min(R, G, B)$$

$$V = M \qquad S = \frac{M - m}{M}$$

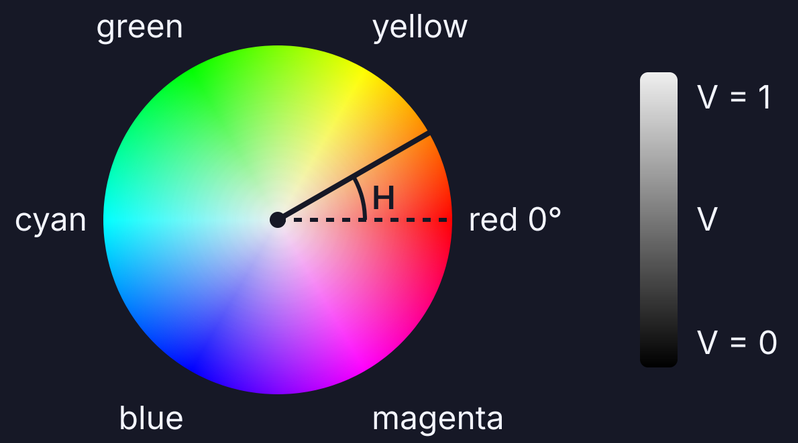
$$H = 60° \times \frac{G - B}{M - m} \quad \text{if } M = R$$

$$H = 60° \times \left(\frac{B - R}{M - m} + 2 \right) \quad \text{if } M = G$$

$$H = 60° \times \left(\frac{R - G}{M - m} + 4 \right) \quad \text{if } M = B$$

What each letter means

- V = the biggest of R, G, B — how bright.
- S = how far the smallest is below the biggest, as a fraction. 0 is grey, 1 is fully vivid.
- H = the angle on the colour wheel. The biggest channel picks the start — red 0°, green 120°, blue 240°. Grey: undefined.



Q1 Colour quick-fire

(a) Magenta in 8-bit RGB?

(b) CMY ink amounts (0–1) for RGB `(0.2, 0.6, 1.0)`? Which ink is not needed?

(c) A colour has hue H = 40°. What is the hue of its complement?

(d) You have 10 bits per pixel. Split them between R, G, B in proportion to luminance `0.2126R + 0.7152G + 0.0722B`.

Q1 Colour quick-fire

(A) MAGENTA

red + blue, no green

(255, 0, 255)

(B) RGB → CMY

C = 1 − R = 0.8
M = 1 − G = 0.4
Y = 1 − B = 0.0

no yellow ink

(C) COMPLEMENT

opposite on the wheel
(40 + 180) mod 360

H = 220°

(D) BIT BUDGET

10 × 0.2126 ≈ 2.1
10 × 0.7152 ≈ 7.2
10 × 0.0722 ≈ 0.7

R 2 · G 7 · B 1

Q2 Three-level posterize

A greyscale image has intensities $0 \le c(x,y) \le 1$. The output may only use black 0, grey 0.5 and white 1.

(a) Write an algorithm that maps each pixel to 0, 0.5 or 1 using two thresholds `lo < hi`.

(b) How would you choose `lo` and `hi` so each level covers roughly a third of the pixels?

(c) A friend fixes `lo = 1/3`, `hi = 2/3`. When does that fail to give equal thirds?

Q2 Three-level posterize

(A) PER PIXEL

```
for each (x, y):
  v = c(x, y)
  if v < lo: out = 0
  elif v < hi: out = 0.5
  else: out = 1
```

`2 comparisons per pixel`

(B) PERCENTILES

```
s = sort(all N pixel values)
lo = s[⌊N/3⌋]
hi = s[⌊2N/3⌋]
```

`33rd and 67th percentile`

(C) FIXED CUT-OFFS

They ignore the histogram. A dark photo with most values below 1/3 comes out almost entirely black.

`depends on the image`

TAKEAWAY Equal counts come from the data — the median for two levels, percentiles for more.

Q3 Region size

A black-and-white image `img(x,y)`, W × H. Pixels are neighbours if they share an edge (4-connected).

(a) Write a recursive `regionSize(a, b)` that returns the number of pixels in the connected region with the same colour as `img(a,b)`.

(b) What goes wrong if you forget to mark pixels as visited?

(c) On a 4000 × 4000 all-white image the program crashes. Why, and how would you fix it?

Q3 Region size

(A) RECURSIVE COUNT

```
int size(x, y, col):
  if x<0 or x≥W or y<0 or y≥H:
    return 0
  if seen[x][y] or img(x,y) ≠ col:
    return 0
  seen[x][y] = true
  return 1 + size(x+1,y,col)
           + size(x−1,y,col)
           + size(x,y+1,col)
           + size(x,y−1,col)

call: size(a, b, img(a,b))
```

(B) NO VISITED MARK

Neighbours call each other back and forth forever — infinite recursion, and every pixel is counted many times.

`mark before recursing`

(C) STACK OVERFLOW

Deep recursion — up to 16 million calls — overflows the stack. A to-do list of pixels in a loop avoids that.

`one sentence is enough`

Q4 Pixel mapping

An image is W wide and H tall. In pixel `(x,y)`, x is the column from the left and y the row from the top, both from 0. Pixels are numbered 0 to W − 1, so the last column is W − 1, not W.

(a) Rotate 90° clockwise into an H × W output. Output pixel `(x,y)` takes the colour of which input pixel?

(b) Mirror left–right, same size. Output `(x,y)` ← which input pixel?

(c) Why loop over output pixels rather than input pixels?

Q4 Pixel mapping

(A) ROTATE 90° CW

forward: in(x,y) → out(H−1−y, x)
check: (0,0) → (H−1, 0) top-right ✓
invert: x = y′, y = H−1−x′

`out(x,y) = in(y, H−1−x)`

(B) MIRROR

column flips, row stays
check: x = 0 ↔ W−1

`out(x,y) = in(W−1−x, y)`

(C) INVERSE MAPPING

Every output pixel gets exactly one value — no holes, no double writes. It matters once the map isn't a clean permutation (scaling, arbitrary angles).

`pull, don't push`

TAKEAWAY The last column is W − 1, not W — that is where every −1 comes from. Check one corner.

Alpha compositing

RGBA

- Alpha = how much of the pixel a layer covers: `1` solid, `0` invisible.

A OVER B — A TAKES ITS SHARE, B SHOWS THROUGH THE GAP

$$\alpha_o = \alpha_a + \alpha_b(1-\alpha_a)$$

$$C_o = \frac{C_a\alpha_a + C_b\alpha_b(1-\alpha_a)}{\alpha_o}$$

- Divide by α_o to get a normal colour back.
- Not symmetric: the top layer wins.



three RGBA layers composited with *over*

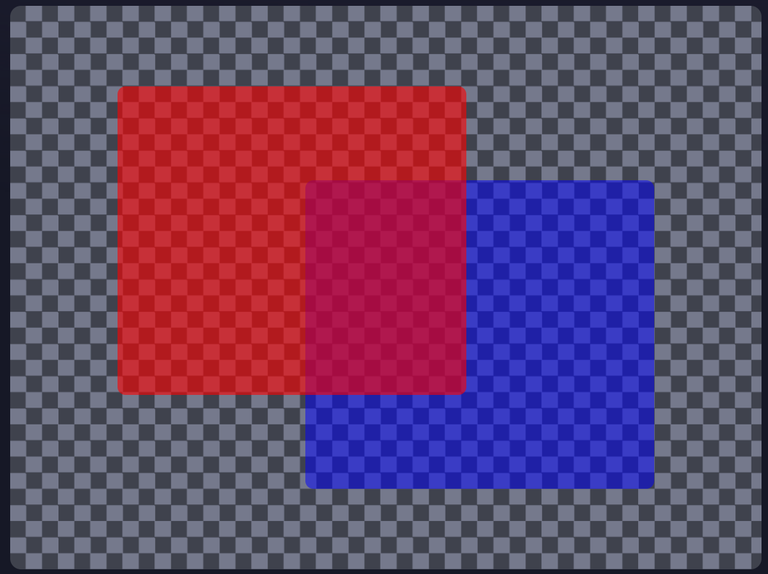
Q5 Compositing

A is red `(1, 0, 0)` with `α = 0.6`. B is blue `(0, 0, 1)` with `α = 0.5`.

(a) Compute α_o and C_o for A over B.

(b) Compute B over A. Is *over* commutative?

(c) If B is opaque, what does A over B simplify to?



A red, α 0.6 — on top B blue, α 0.5

Q5 Compositing

(A) A OVER B

$\alpha_o = 0.6 + 0.5 \times 0.4 = 0.8$

$C_o = (0.6 \cdot (1,0,0) + 0.2 \cdot (0,0,1)) / 0.8$

 $(0.75, 0, 0.25)$

(B) B OVER A

$\alpha_o = 0.5 + 0.6 \times 0.5 = 0.8$

$C_o = (0.5 \cdot (0,0,1) + 0.3 \cdot (1,0,0)) / 0.8$

 $(0.375, 0, 0.625)$

(C) B OPAQUE ($\alpha_b = 1$)

$\alpha_o = \alpha_a + (1 - \alpha_a) = 1$

no division needed

$\alpha_a C_a + (1 - \alpha_a) C_b$

TAKEAWAY Output alpha is symmetric; colour is not — the top layer dominates. Over an opaque background, *over* is a linear blend.

Q6 Stamp composite

A scan `S` is black ink on white paper, greyscale 0–1. `P` is a photo of a parcel, channels in [0, 1].

(a) Give a per-pixel operator that prints the stamp onto P: the paper vanishes, the ink darkens the photo.

(b) Change it so the ink shows as a colour K (say dark red) instead of black. Hint: treat `1 − S` as alpha.

(c) What does (b) give for pure paper? For pure ink?

Q6 Stamp composite

(A) MULTIPLY

paper S = 1 → P unchanged
ink S = 0 → black

$B = P \cdot S$ (per channel)

(B) INK OVER PHOTO

$\alpha = 1 - S$ (ink coverage)
K over P

$B = \alpha \cdot K + (1 - \alpha) \cdot P$

(C) CHECK THE ENDS

S = 1: α = 0 → B = P
S = 0: α = 1 → B = K

paper → P · ink → K

TAKEAWAY No alpha channel? Derive one from the image — here, darkness is coverage.

Gamma, banding, clipping

GAMMA — SCREENS AREN'T LINEAR

$$I = a^{\gamma} \qquad \gamma = \frac{\ln 0.5}{\ln a_{1/2}}$$

- γ > 1 darkens the middle; 0 and 1 never move. Measure γ from $a_{1/2}$, the value that looks half as bright.

WHY GAMMA — BANDING

- Only 256 levels, and eyes spot steps in the darks — so gamma puts more levels there.
- The screen's curve turns the stored number back into light.

CLIPPING

- Values past the max get cut off — detail is lost.



24 levels stored as linear light — banding in the darks



the same 24 levels gamma-encoded — steps look even

Q7 Two displays

Two displays share the same maximum brightness. Intensity is $I = a^{\gamma}$ for pixel value $0 \le a \le 1$.

(a) Display A has $\gamma = 2$. A pixel value of 0.5 looks four times dimmer on display B than on A. What is γ_B?

(b) Which pixel value on B matches the brightness of 0.5 on A?

(c) Which pixel values look identical on both displays, whatever their gammas?

Q7 Two displays

(A) SOLVE FOR Γ

$I_A = 0.5^2 = 0.25$

$I_B = 0.25 / 4 = 0.0625$

$0.5^{\gamma} = 0.0625 = 0.5^4$

$\gamma_B = 4$

(B) MATCH BRIGHTNESS

$a^4 = 0.25$

$a = 0.25^{(1/4)} = \sqrt{0.5}$

$a \approx 0.707$

(C) FIXED POINTS

$0^{\gamma} = 0$

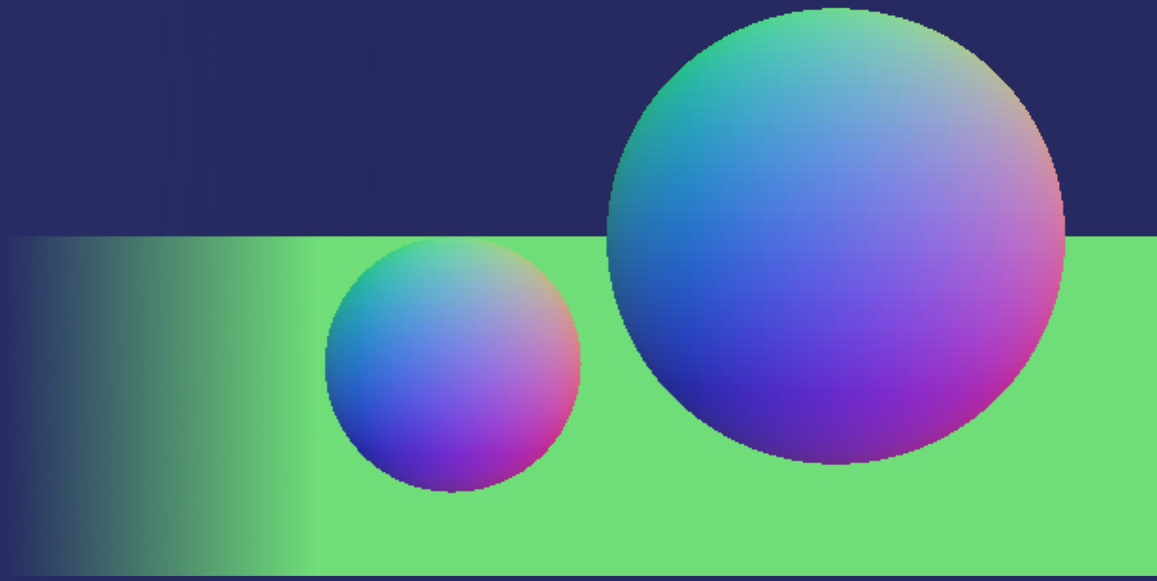
$1^{\gamma} = 1$

0 and 1 only

TAKEAWAY Gamma bends the middle of the range; black and white stay pinned.

02

Ray casting



Rasterization vs ray casting

Rasterization — each object → its pixels

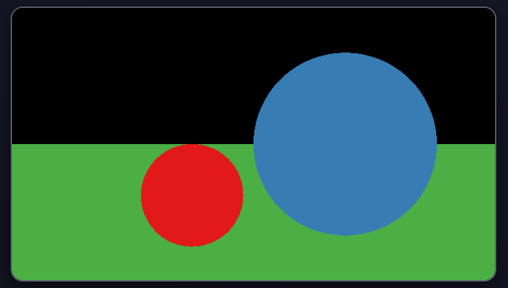
```
for each scene object:
    for each image pixel:
        if object affects pixel: do something
```

— Project corners, fill the pixels inside. Fast on GPUs.

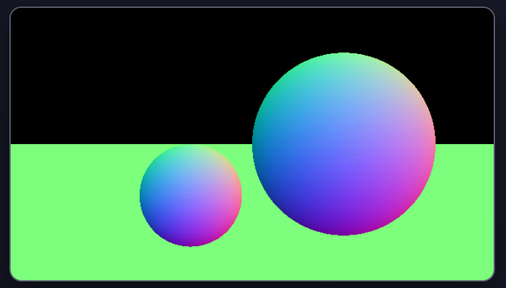
Ray casting — each pixel → its object

```
for each image pixel:
    generate a ray
    for each scene object:
        if ray intersects object: set
pixel colour
```

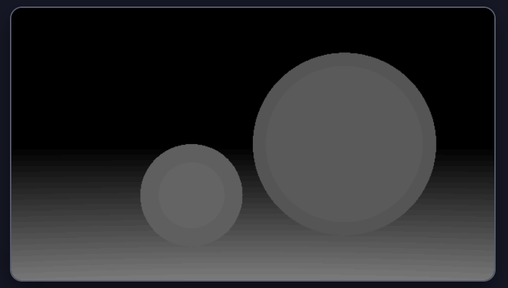
— Ray per pixel, find the first hit. Loops swapped.



object ID



surface normal

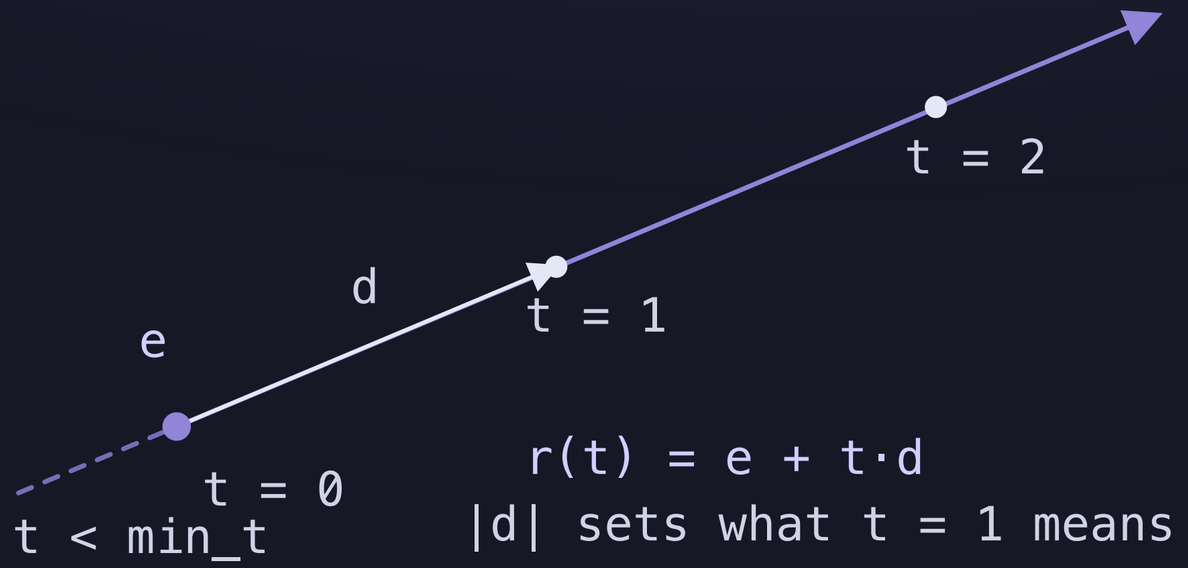


depth

A ray is a point and a direction

ONE PARAMETER, T

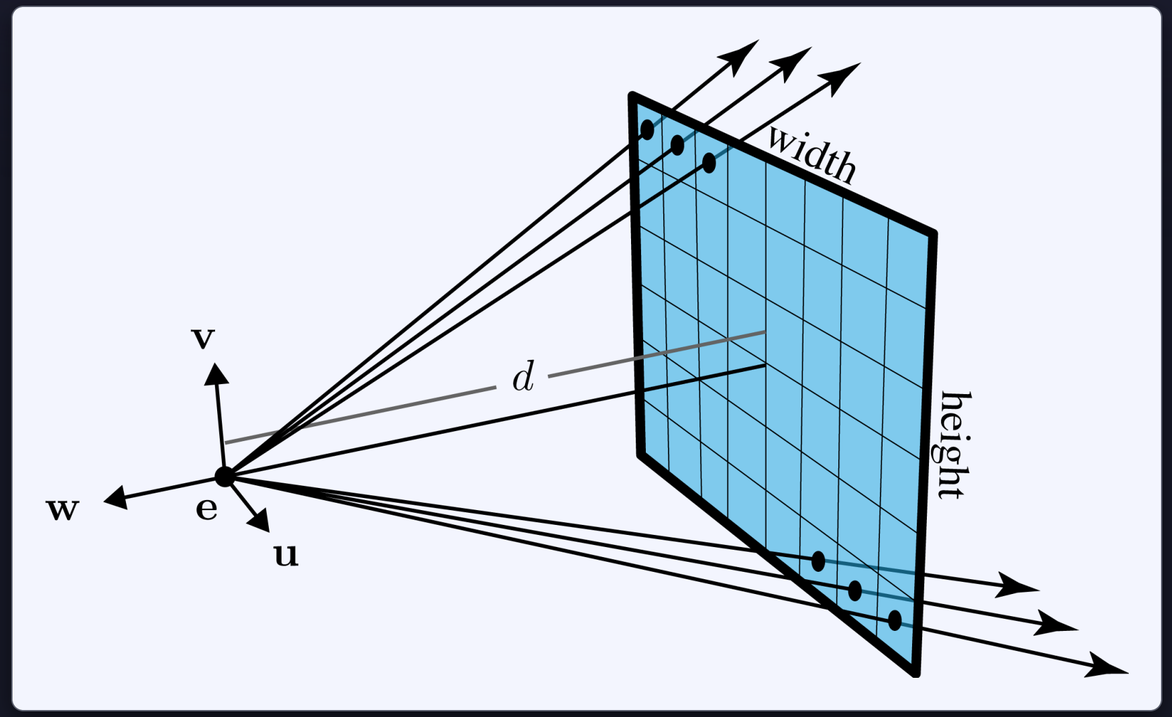
- `r(t) = e + t·d`: a start point plus a direction. Each `t` is one point along the line.
- **`d` is *not* normalised**: on purpose — its length decides what `t = 1` means: the screen, or the light.



The camera

PIXEL CONVENTIONS

- The test uses the textbook: `i` = column from the left, `j` = row from the *bottom*.

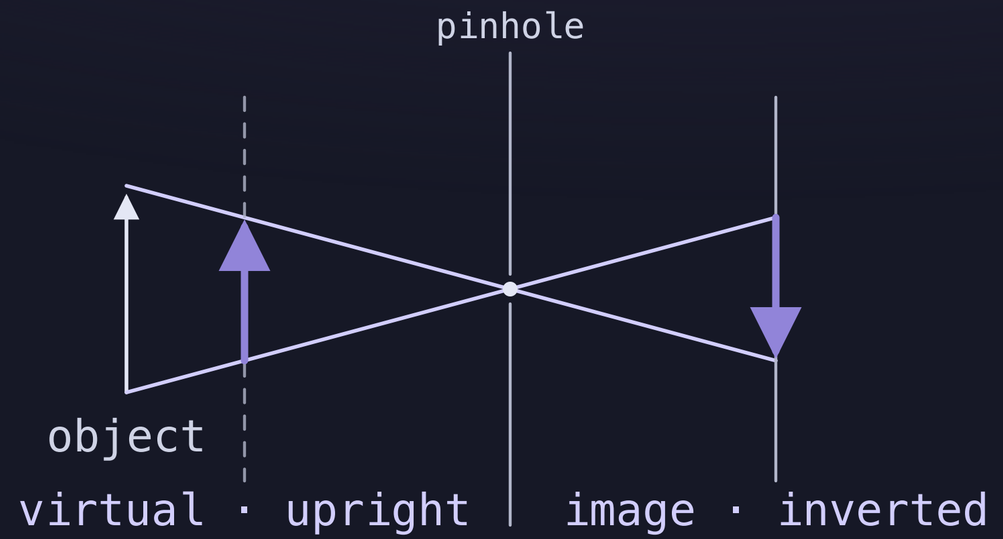


the eye e is the pinhole; the screen floats d in front of it, cut into pixels

Camera models

FROM A DARK ROOM TO OUR VIRTUAL CAMERA

1. **Camera obscura** — a dark room with a small hole: the far wall shows the outside, upside down.
2. **Ideal pinhole** — a tiny hole: each scene point lands on exactly one spot. Sharp everywhere.
3. **Real pinhole** — a bigger hole blurs each point into a disc. Smaller is sharper, but darker.
4. **Lens** — lets in lots of light and focuses it, but is sharp at one distance only.
5. **Our camera** — put the screen *in front of* the eye, not behind: same picture, right way up.



similar triangles: image height = object height × (image distance / object distance)

The whole idea — one ray, gathering colour



× 0.5 × 0.3 × 0.2

\+ \+ = one pixel

Pixel → camera → world

1 · PIXEL

`(i, j)`

which pixel: column from the left, row from the bottom

2 · CAMERA SPACE

$(x_c, y_c, -d)$

where its centre sits on the screen, d in front of the eye

3 · WORLD SPACE

`e + t·(…)`

turn it into a world direction with u, v, w; start at the eye

$$x_c = -W/2 + W\,\frac{i + 0.5}{n_x} \qquad y_c = -H/2 + H\,\frac{j + 0.5}{n_y}$$

$$p(t) = e + t\,(x_c\,\mathbf{u} + y_c\,\mathbf{v} - d\,\mathbf{w})$$

WATCH FOR `+ 0.5` hits the pixel centre, not its corner. Perspective: one start, many directions. Orthographic: one direction, many starts.

Q8 Generating a ray

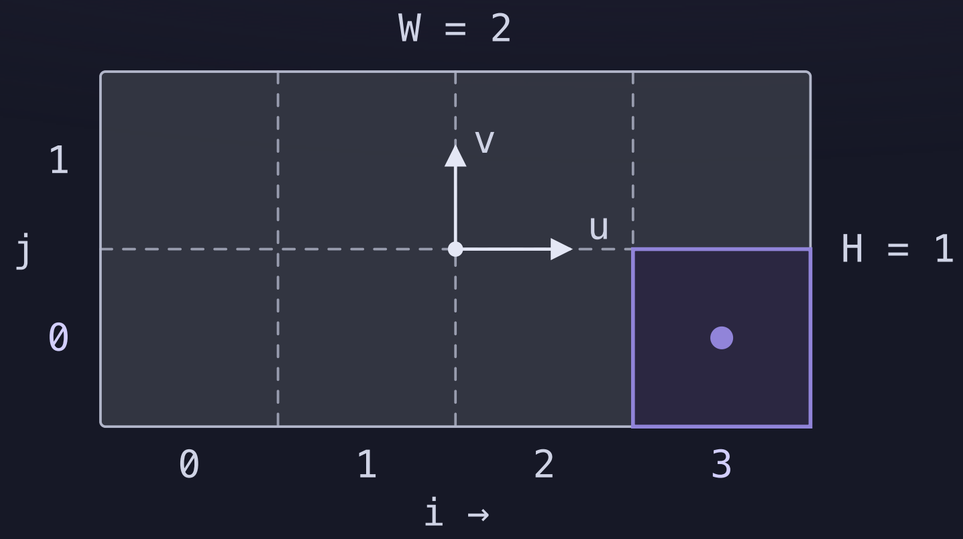
Camera at `e = (1, 2, 3)`, `view = (0, 0, −1)`, `up = (0, 1, 0)`. Image 4 × 2 pixels; image plane `W = 2`, `H = 1`; `d = 1`.

(a) Compute u, v, w.

(b) Camera-space coordinates of the centre of pixel `(3, 0)`.

(c) The world-space perspective ray through that pixel.

(d) The orthographic ray through the same pixel.



the image plane, seen from the eye · origin at its centre

Q8 Generating a ray

(A) CAMERA FRAME

$w = -\text{view} = (0, 0, 1)$

$u = (0,0,-1) \times (0,1,0) = (1, 0, 0)$

$v = w \times u = (0, 1, 0)$

$u\ (1,0,0) \cdot v\ (0,1,0) \cdot w\ (0,0,1)$

(B) PIXEL CENTRE

$x_c = -1 + 2 \times 3.5 / 4 = 0.75$

$y_c = -0.5 + 1 \times 0.5 / 2 = -0.25$

$(0.75, -0.25)$

(C) PERSPECTIVE

origin e, direction $x_c u + y_c v - d \cdot w$

$p(t) = (1,2,3) + t(0.75, -0.25, -1)$

(D) ORTHOGRAPHIC

origin $e + x_c u + y_c v$, direction $-w$

$p(t) = (1.75, 1.75, 3) + t(0, 0, -1)$

TAKEAWAY Perspective varies the *direction* per pixel; orthographic varies the *origin*.

Q9 Rays quick-fire

(a) A line passes through A = `[2, −1, 3]` at t = 0 and B = `[8, 2, 0]` at t = 3. Where is it at t = 4?

(b) The ray `[0,0,0] + t[1,2,2]` meets the plane z = 6. Find t and the point.

(c) Minimum number of floats to specify a solid finite cylinder in 3D, any orientation?

(d) Orthographic vs perspective camera: across pixels, which varies — the ray origin or its direction?

Q9 Rays quick-fire

(A) POINT AT T = 4

```
d = (B − A) / 3
  = [2, 1, −1]
A + 4d
```

`[10, 3, −1]`

(B) RAY–PLANE

```
z: 2t = 6
t = 3
```

`[3, 6, 6]`

(C) CYLINDER

```
end centres 3 + 3
radius 1
(or centre 3, axis 2,
length 1, radius 1)
```

`7`

(D) CAMERAS

Orthographic: origin moves across the image plane, direction fixed at −w.
Perspective: origin fixed at the eye, direction varies.

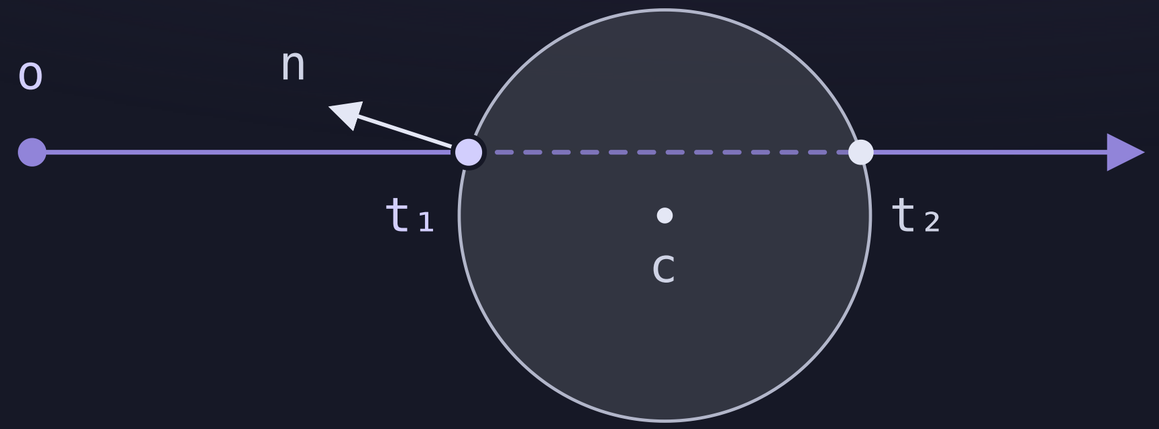
sphere_intersect

THE IDEA

- Plug the ray into $|q - c|^2 = r^2$ → a quadratic in t: 0, 1 or 2 answers.

HOW TO COMPUTE IT

- `sphere_intersect(center, radius, ray, min_t)` → `(hit, t, n)`. Negative discriminant: miss. Otherwise pick the root as shown.



Take the smaller `t`. Too small? You're inside — take the bigger. Both too small: it's behind you, miss.

```
# c=(0,0,0) r=0.5
A=9 B=-30 C=24.75
disc=9 → t=1.5 ✓
hit (0,0,0.5)
n=(0,0,1)
```



`inside-a-sphere` — the far root

plane_intersect

THE IDEA

- A plane is a point p_0 and a normal `n`. Plugging in the ray gives a simple linear equation — at most one hit.

HOW TO COMPUTE IT

- `plane_intersect(point, normal, ray, min_t)` → `(hit, t, n)`. Ray parallel to the plane (`d·n = 0`): no hit. Reject `t < min_t`.



At most one hit: `t = −((o − p₀)·n) / (d·n)`. `d·n = 0` → parallel, no hit.

```
# floor: p₀=(0,−0.5,0)  n=(0,1,0)
axis ray d=(0,0,−3):  d @ n = 0  → miss
tilt d=(0,−0.2,−3):   t = −0.5 / −0.2 = 2.5
```

triangle_intersect

THE IDEA

- Any point on the triangle is corner `a`, plus some of edge 1 (β), plus some of edge 2 (γ). Set it equal to the ray, solve for `β, γ, t`.

HOW TO COMPUTE IT

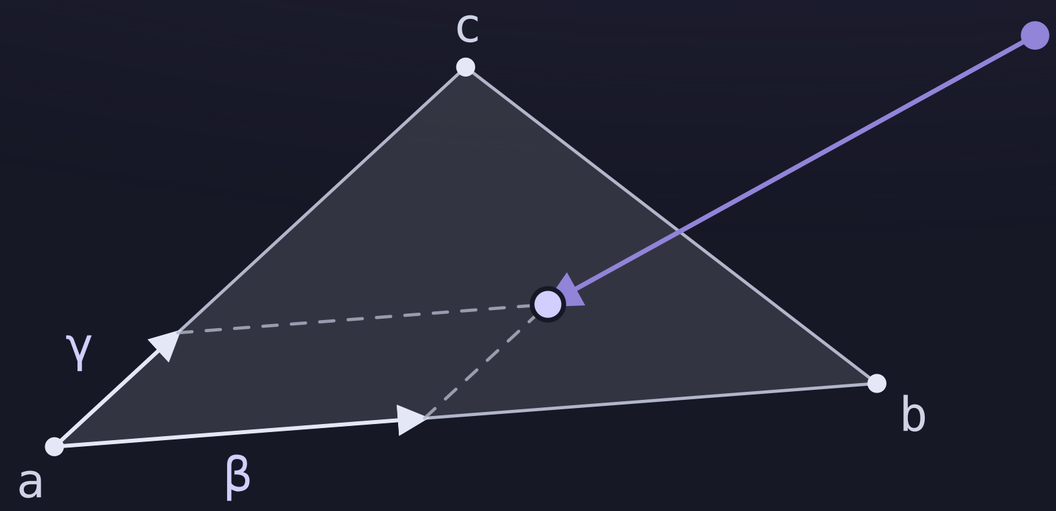
- `triangle_intersect(a, b, c, ray, min_t)` → `(hit, t, n)` by Cramer's rule. Normal: `normalize((a−b) × (a−c))` — this order.

CRAMER'S RULE

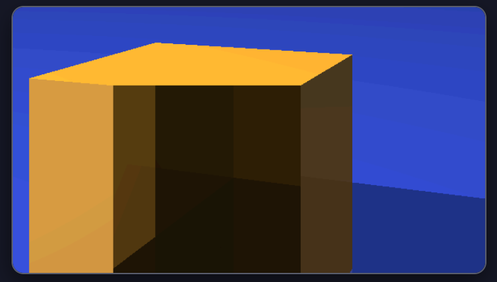
$$Ax = b$$

$$x_i = \frac{\det A_i}{\det A}$$

- A_i is `A` with its `i`-th column replaced by `b`.



Inside when `β ≥ 0`, `γ ≥ 0` and `β + γ ≤ 1` — and `t ≥ min_t`.



`cube-and-plane` — twelve triangles from `cube.stl`

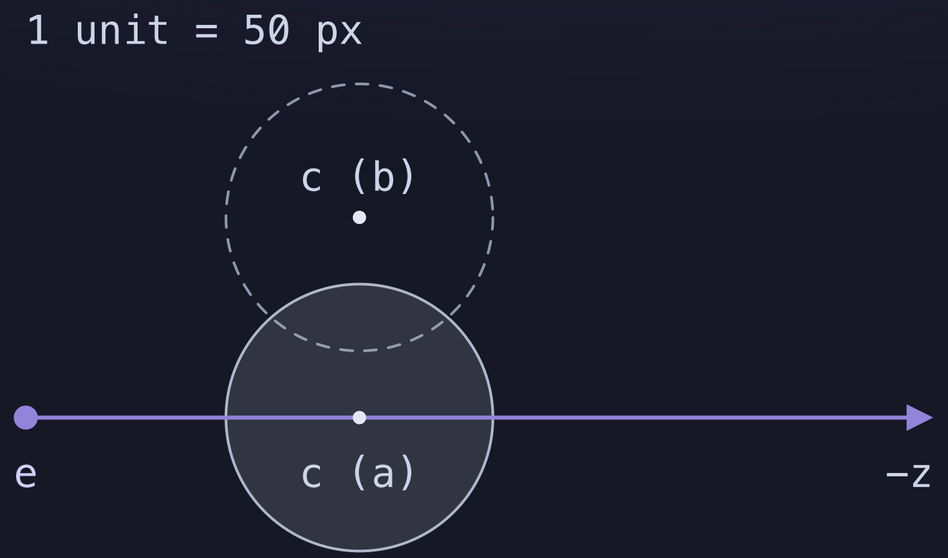
Q10 Plane and sphere

One ray for every part: $e = (0, 0, 0)$, $d = (0, 0, -1)$.

(a) Sphere $c = (0, 0, -5)$, $r = 2$: both t values, the visible hit point and its normal.

(b) Sphere $c = (3, 0, -5)$, $r = 2$: does the ray hit?

(c) Plane through $p_0 = (0, 0, -4)$ with $n = (0, 0, 1)$: find t. What if $n = (1, 0, 0)$?



top view: sphere (a) on the ray, sphere (b) three units to the side (+x)

Q10 Plane and sphere

(A) SPHERE ON THE RAY

$A = 1$

$B = 2(0,0,-1)\cdot(0,0,5) = -10$

$C = 25 - 4 = 21$

$\Delta = 100 - 84 = 16$

$t = (10 \mp 4) / 2 = 3, 7$

$p = e + 3d,\ n = (p - c)/r$

$p\ (0,0,-3) \cdot n\ (0,0,1)$

(B) SPHERE OFF THE RAY

$e - c = (-3, 0, 5)$

$B = -10$

$C = 9 + 25 - 4 = 30$

$\Delta = 100 - 120 = -20$

$\Delta < 0 \rightarrow$ miss

(C) PLANE

$t = (p_0 - e)\cdot n / (d\cdot n)$

$= (0,0,-4)\cdot(0,0,1) / -1 = 4$

$n = (1,0,0)$: plane $x = 0$

$d\cdot n = 0$ and $(p_0 - e)\cdot n = 0 \rightarrow 0/0$

$t = 4$ · then ray lies in plane → no hit

TAKEAWAY The smallest positive root is the visible hit. In (b) the ray passes 3 units from the centre — more than the radius 2.

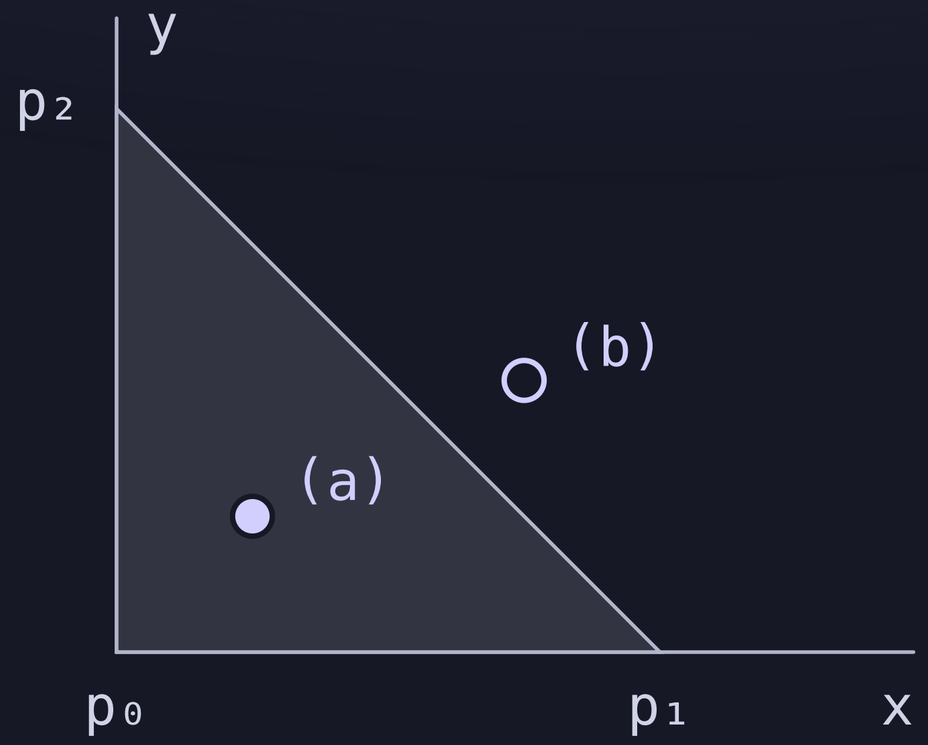
Q11 Triangle

Triangle $p_0 = (0, 0, 0)$, $p_1 = (2, 0, 0)$, $p_2 = (0, 2, 0)$.
Ray direction $d = (0, 0, -1)$.

(a) From $e = (0.5, 0.5, 2)$: solve for β, γ, t. Hit?

(b) From $e = (1.5, 1, 2)$: hit or miss? Which condition decides?

(c) The normal $(p_1 - p_0) \times (p_2 - p_0)$. Does it face the ray?



top view · both rays point straight down, into the page

Q11 Triangle

(A) INSIDE

$(0.5, 0.5, 2 - t) = (2\beta, 2\gamma, 0)$

$\beta = 0.25$ $\gamma = 0.25$ $t = 2$

$\beta + \gamma = 0.5 \leq 1$

hit at $(0.5, 0.5, 0)$

(B) OUTSIDE

$\beta = 0.75$ $\gamma = 0.5$

$\beta, \gamma \geq 0$ ✓

$\beta + \gamma = 1.25 > 1$ ✗

miss — past edge p_1p_2

(C) NORMAL

$(2,0,0) \times (0,2,0) = (0,0,4)$

$n = (0, 0, 1)$

$n \cdot d = -1 < 0$

faces the ray

TAKEAWAY $\beta + \gamma = 1$ is the edge opposite p_0. Swapping the cross-product order flips the normal.

Q12 Dome intersection

A solid dome: the half of a sphere of radius r at the origin with y ≥ 0, closed by a flat disc in the XZ plane. Ray `S + tD`, D unit. Given: `int sphereIntersect(S, D, C, r, float t[])` — returns # hits, t sorted, t > 0.

(a) Write `int XZplaneIntersect(S, D, float t[])`.

(b) Write `int domeIntersect(S, D, r, float t[])`, returning all hits with t > 0 in increasing order.

(c) What is the maximum number of intersections? Why?

Q12 Dome intersection

(A) XZ PLANE

```
if D.y == 0: return 0
t0 = -S.y / D.y
if t0 ≤ 0: return 0
t[0] = t0
return 1
```

(B) DOMEINTERSECT

```
n = 0
k = sphereIntersect(S,D,0,r,ts)
for i in 0..k-1:
  if (S + ts[i]·D).y ≥ 0:
    t[n++] = ts[i]
if XZplaneIntersect(S,D,tp):
  P = S + tp[0]·D
  if P.x² + P.z² < r²:
    t[n++] = tp[0]
sort(t, n); return n
```

(C) MAXIMUM

The dome is convex: a straight line enters once and leaves once. A graze along the rim counts as one.

2

TAKEAWAY Intersect each surface, keep hits on the part that exists, then sort.

Q13 Bounding volumes

Points $P_1, \ldots, P_n$ in 3D.

(a) Give an O(n) algorithm for the tightest axis-aligned bounding box.

(b) Build a bounding sphere from the box. Is it always the tightest sphere?

(c) Give a point set where the box has the smaller volume, and one where the sphere does.

(d) Why wrap a 10,000-triangle mesh in either before ray tracing?

Q13 Bounding volumes

(A) BOX

```
lo = hi = P₁
for each Pᵢ:
  lo = min(lo, Pᵢ)
  hi = max(hi, Pᵢ)
(per component)
```

box [lo, hi]

(B) SPHERE

```
c = (lo + hi) / 2
r = maxᵢ |Pᵢ − c|
```

valid, not minimal

(C) COMPARE

```
box wins: (0,0,0),
(10,1,1)
  10 vs r≈5.05 → 540
sphere wins: (±1,0,0),
  (0,±1,0),(0,0,±1)
  8 vs 4.19
```

(D) WHY

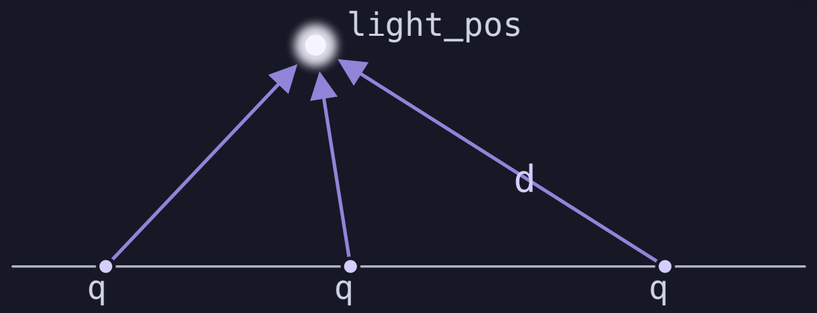
One cheap ray–box or ray–sphere test rejects every ray that misses, skipping 10,000 triangle tests.

03

Ray tracing



Two light helpers — which way is the light, and how far



`d = light_pos − q`
`each d points at the light · |d| varies`

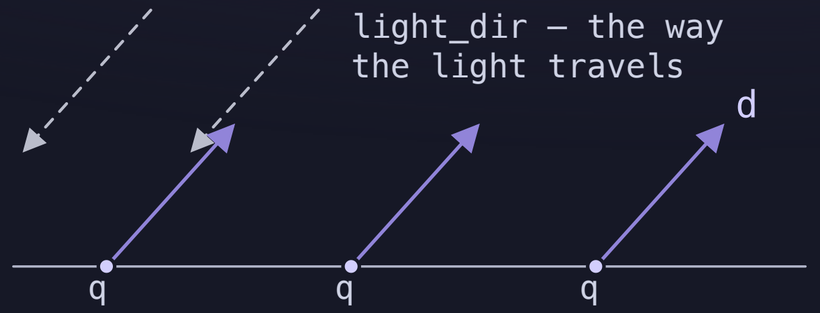
`point_light_direction`

THE IDEA

- A bulb: it has a position, so the direction depends on *where you are*.

HOW TO COMPUTE IT

- `(light_pos − q, 1.0)` — *not* normalised, so `t = 1` lands exactly on the light.



`d = −normalize(light_dir)`
`d points back along light_dir · max_t = ∞`

`directional_light_direction`

THE IDEA

- The sun: so far away every point sees the *same* direction. `max_t = ∞`.

HOW TO COMPUTE IT

- `light_dir` is the way the light *travels*, so toward the light is `−light_dir`.

Lights and normals

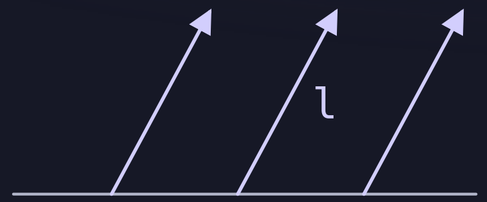
LIGHT DIRECTION L

- *Directional*: the same `l` everywhere — the light is very far away.
- *Point*: `l` points to the light; falls off as $1/r^2$.

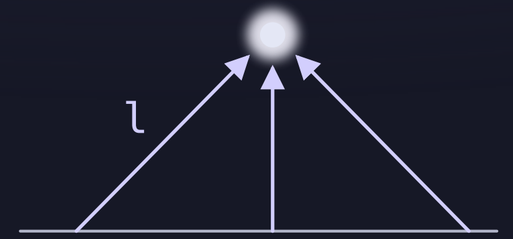
THE NORMAL — THE WAY THE SURFACE FACES

- Flat polygon: cross product of two edges.
- Sphere, or any `f(p) = 0`: the gradient — for a sphere, centre out through the hit.
- Parametric `f(s, t)`: cross the two tangents.

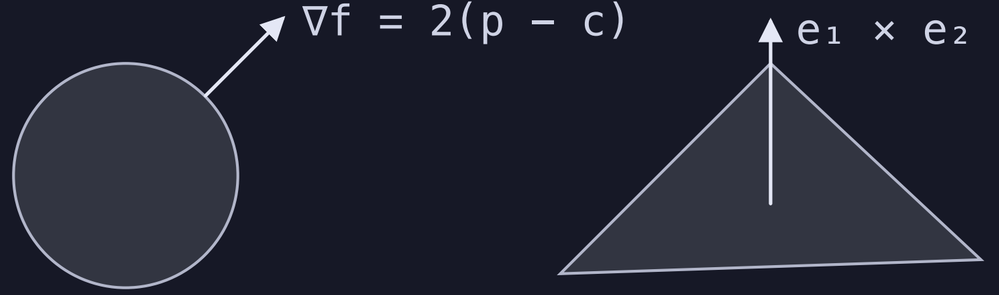
directional



point



normals



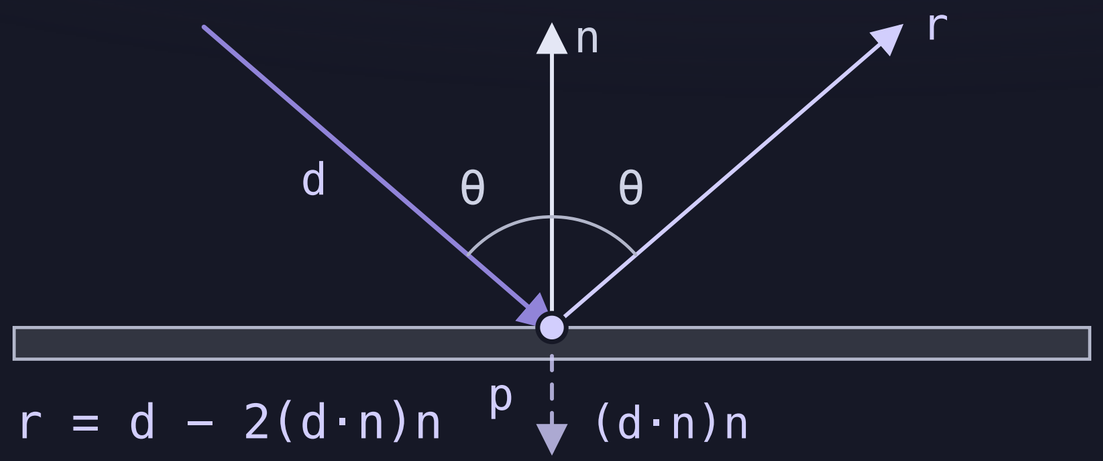
reflect

THE IDEA

- Flip the part of `d` that points into the surface, keep the rest. That part is `(d·n)n` — subtract it **twice**.

HOW TO COMPUTE IT

- `reflect(d, n)` → `d − 2 (d·n) n`. The length doesn't change — don't normalise.

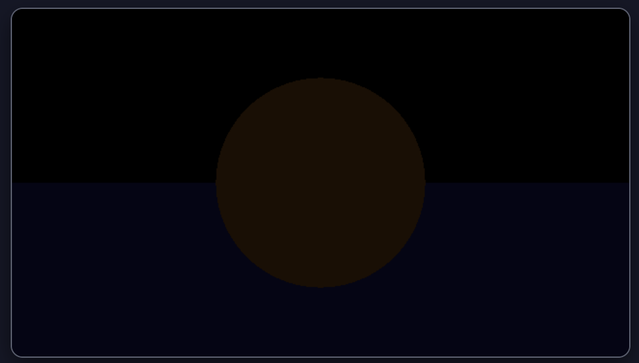


Same angle in, same angle out. Shading writes it `r = 2(n·l)n − l` — the same thing, with `l = −d` pointing away from the surface.

```
# running example · the sphere's front point
d=(0,0,−3) n=(0,0,1) d@n=−3 → r=(0,0,3)
```

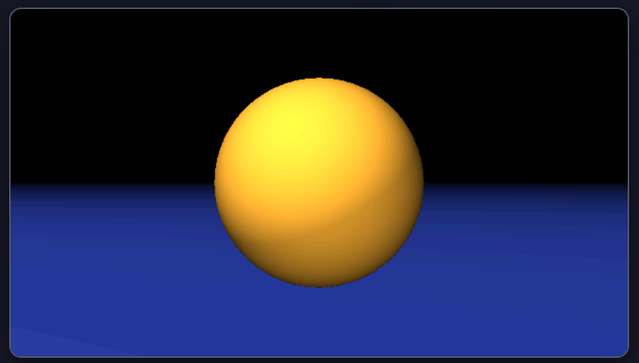
blinn_phong_shading — three terms

`L += 0.1 · ka`



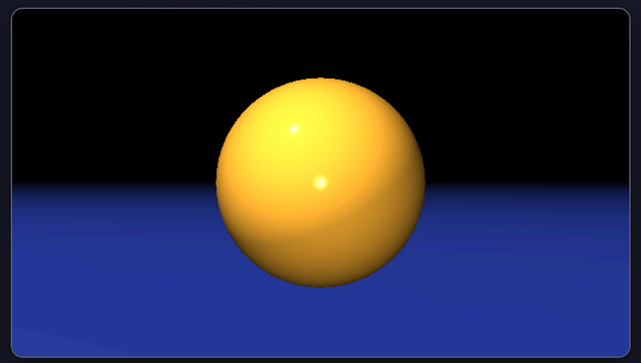
ambient — added once; fakes bounced light

`L += I · kd · max(0, n·l)`



+ diffuse — slanted light spreads thinner

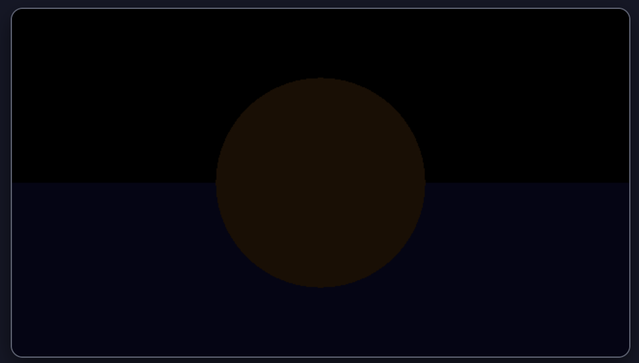
`L += I · ks · max(0, n·h)^p`



+ specular — p bigger = tighter, not brighter

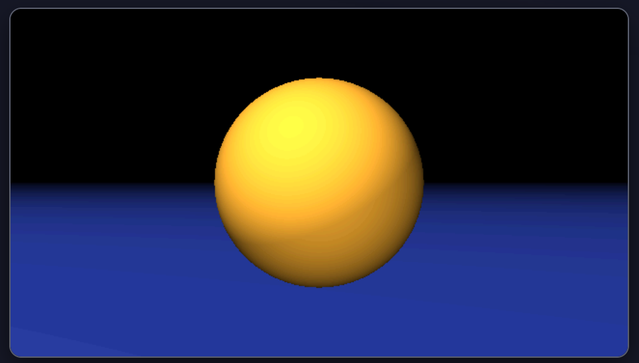
blinn_phong_shading — the two factors, up close

`L += 0.1 · ka`

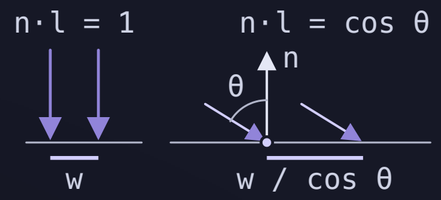


ambient — added once; fakes bounced light

`L += I · kd · max(0, n·l)`

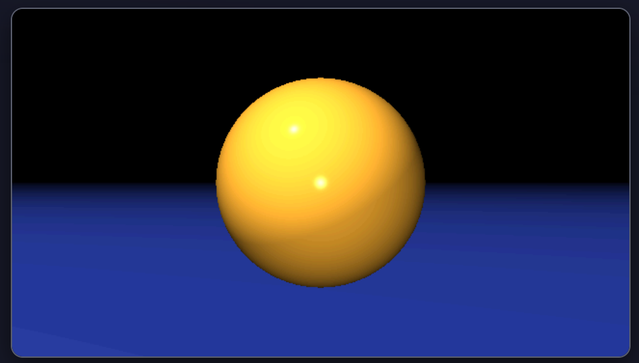


+ diffuse — slanted light spreads thinner

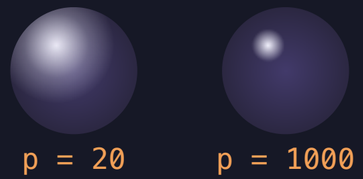


cosine foreshortening · `n·l = cos θ`

`L += I · ks · max(0, n·h)^p`



+ specular — p bigger = tighter, not brighter



phong_exponent · bigger p, tighter dot

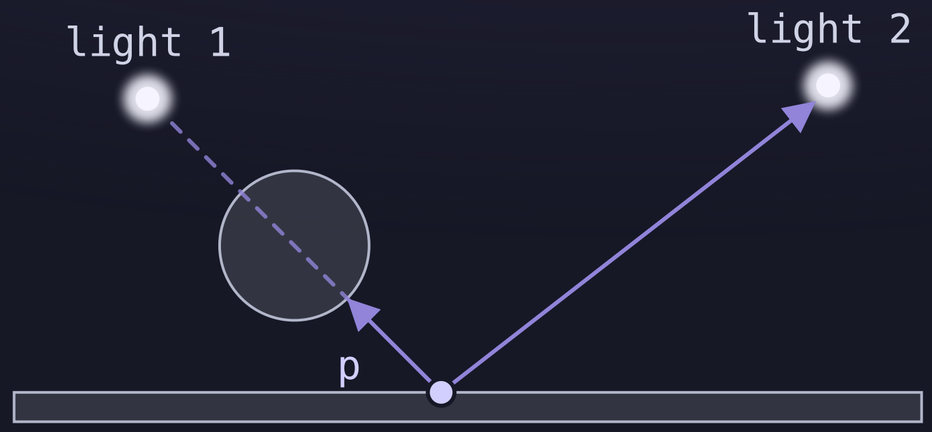
blinn_phong_shading

HOW TO COMPUTE IT

`blinn_phong_shading(ray, hit_id, t, n, objects, lights)` → `rgb`: add up the light from every lamp the point can see.

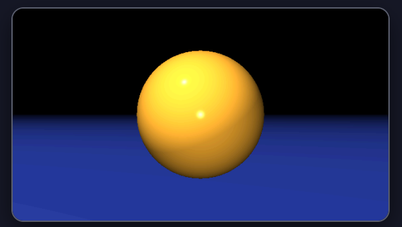
ALGORITHM

1. Hit point `p = ray.origin + t · ray.direction`.
2. Ambient, added **once**: `0.1 · m.ka` — `m` is the hit material.
3. For each light, `light_direction(light, p)` → `(d, max_t)`: toward the light, and how far.
4. Shadow ray from `p` along `d`, offset `1e–6`. Anything before `max_t` → blocked, skip that light.
5. Otherwise add diffuse `m.kd · I · max(0, n·l)` and specular `m.ks · I · max(0, n·h)^p`.
6. Return the sum **un-clamped**.

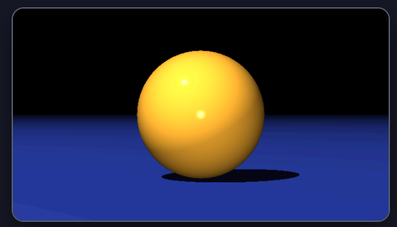


`blocked → skip light 1  light 2 counts`

The epsilon: start the shadow ray a tiny bit (1e–6) off the surface, or it hits itself at `t ≈ 0` — black speckles.

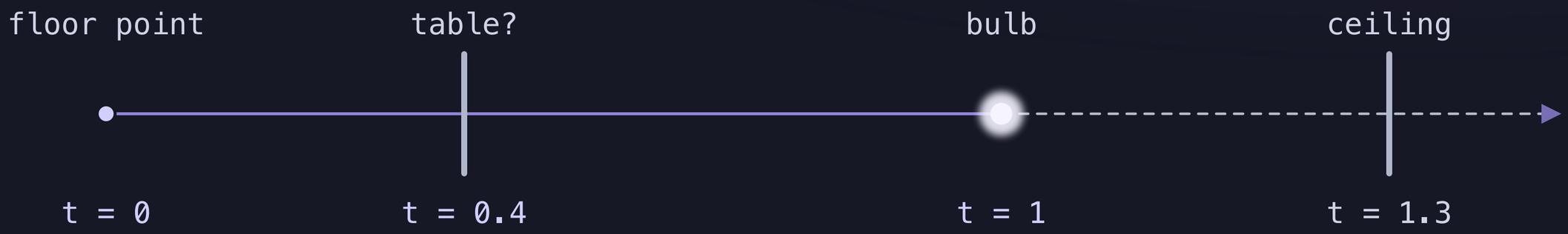


before — no shadow test



after — with shadows

Why only 0 < t < 1 casts a shadow



HIT AT T = 0.4 — TABLE

Between the floor and the bulb. It blocks the light: **shadow**.

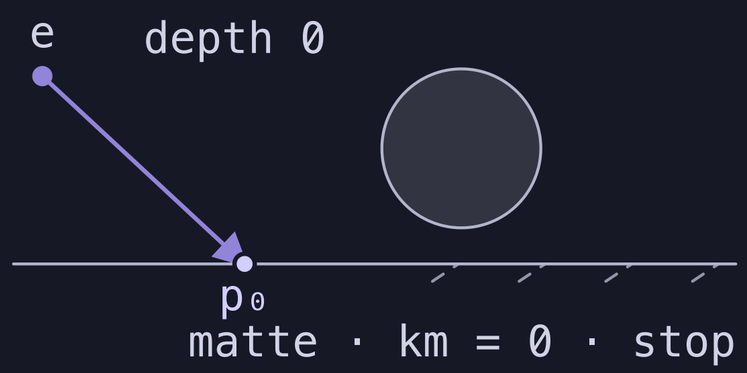
HIT AT T = 1.3 — CEILING

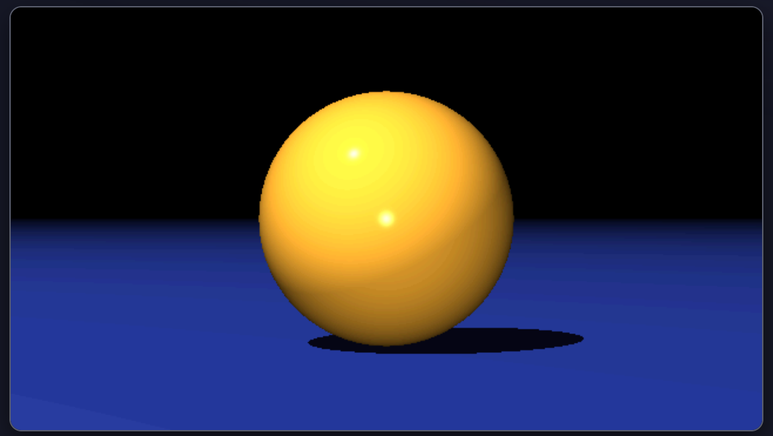
Past the bulb. It can't block anything: **lit**.

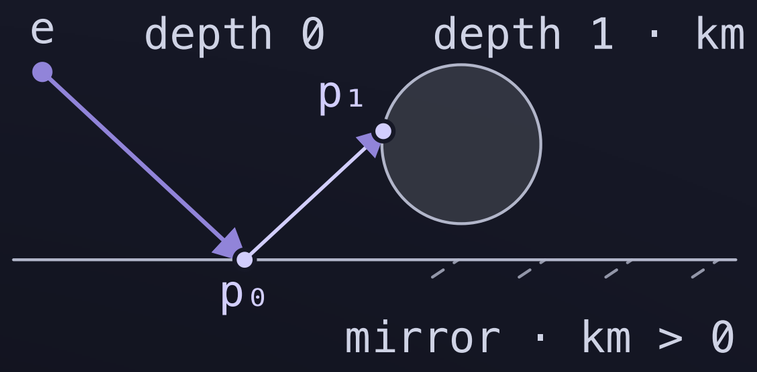
TAKEAWAY `d = light − q`, so t = 1 is the bulb — the finish line. Only hits before it cast a shadow.

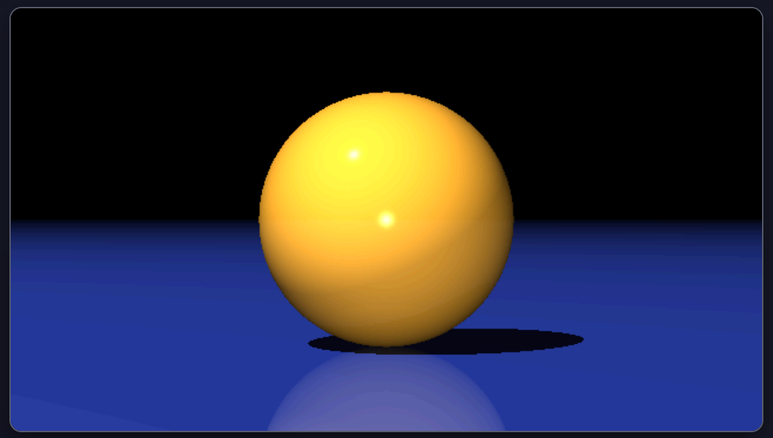
raycolor — shading, plus one bounce deeper

THE IDEA









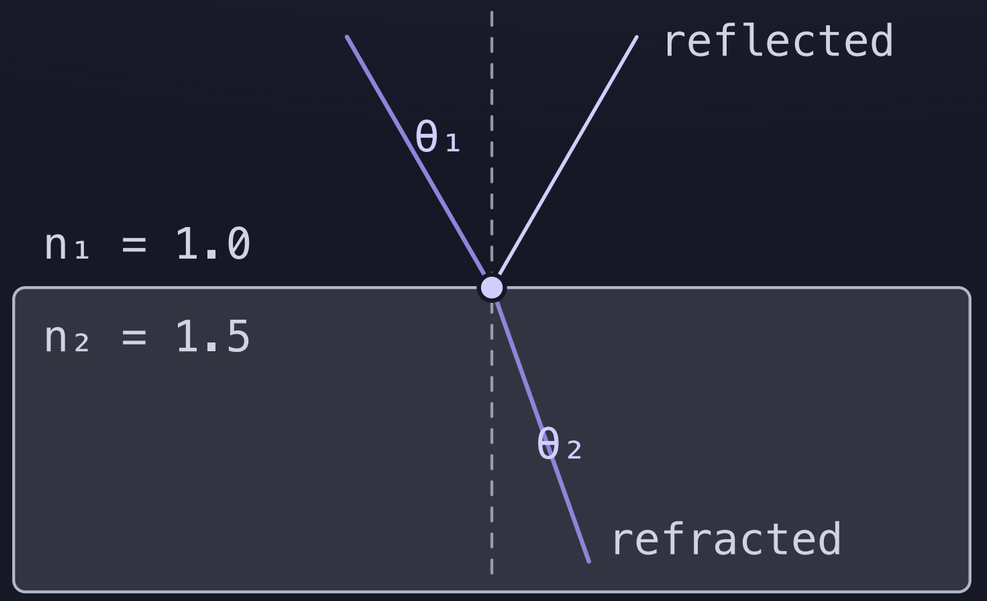
Recursion and refraction

```
def rayTrace(ray, depth):
    if ray hits an object:
        colour = shading model
        if depth < maxDepth:
            colour += rayTrace(reflected, depth+1)
            colour += rayTrace(refracted, depth+1)
    return colour
```

SNELL'S LAW

$$n_1 \sin \theta_1 = n_2 \sin \theta_2$$

- Mirrors and glass shoot one more ray and add its colour — each bounce can double the rays. Into glass bends toward the normal; out, away.



Q14 True or false

The reason earns the marks. V, L, N unit; R is L reflected about N; `H = normalize(L + V)`.

(a) The specular highlight on a shiny sphere stays at the same surface point as the viewer walks around it.

(b) For any L and V, Phong and Blinn highlights are centred on the same surface point.

(c) A ray leaving glass (n = 1.5) into air bends toward the normal.

(d) A desk lamp 30 cm above a keyboard is well modelled as a directional light.

(e) Doubling the shininess exponent doubles the peak brightness of the highlight.

Q14 True or false

(A) FALSE

Specular depends on V through V·R or N·H — move the eye and the peak moves. Only diffuse is view-independent.

(B) TRUE

$V \cdot R = 1 \Leftrightarrow R = V \Leftrightarrow N$ bisects L and $V \Leftrightarrow N = H \Leftrightarrow N \cdot H = 1$. Only the falloff differs.

(C) FALSE

$1.5 \sin \theta_1 = \sin \theta_2$, so $\theta_2 > \theta_1$: it bends away. Past $\theta_1 \approx 41.8°$: total internal reflection.

(D) FALSE

L and distance change a lot across the keyboard — use a point or spot light. Directional suits the sun.

(E) FALSE

At $N \cdot H = 1$, $(N \cdot H)^p = 1$ for any p; the peak stays $k_s I$. Larger p only narrows it.

Antialiasing and soft shadows

SUPERSAMPLING

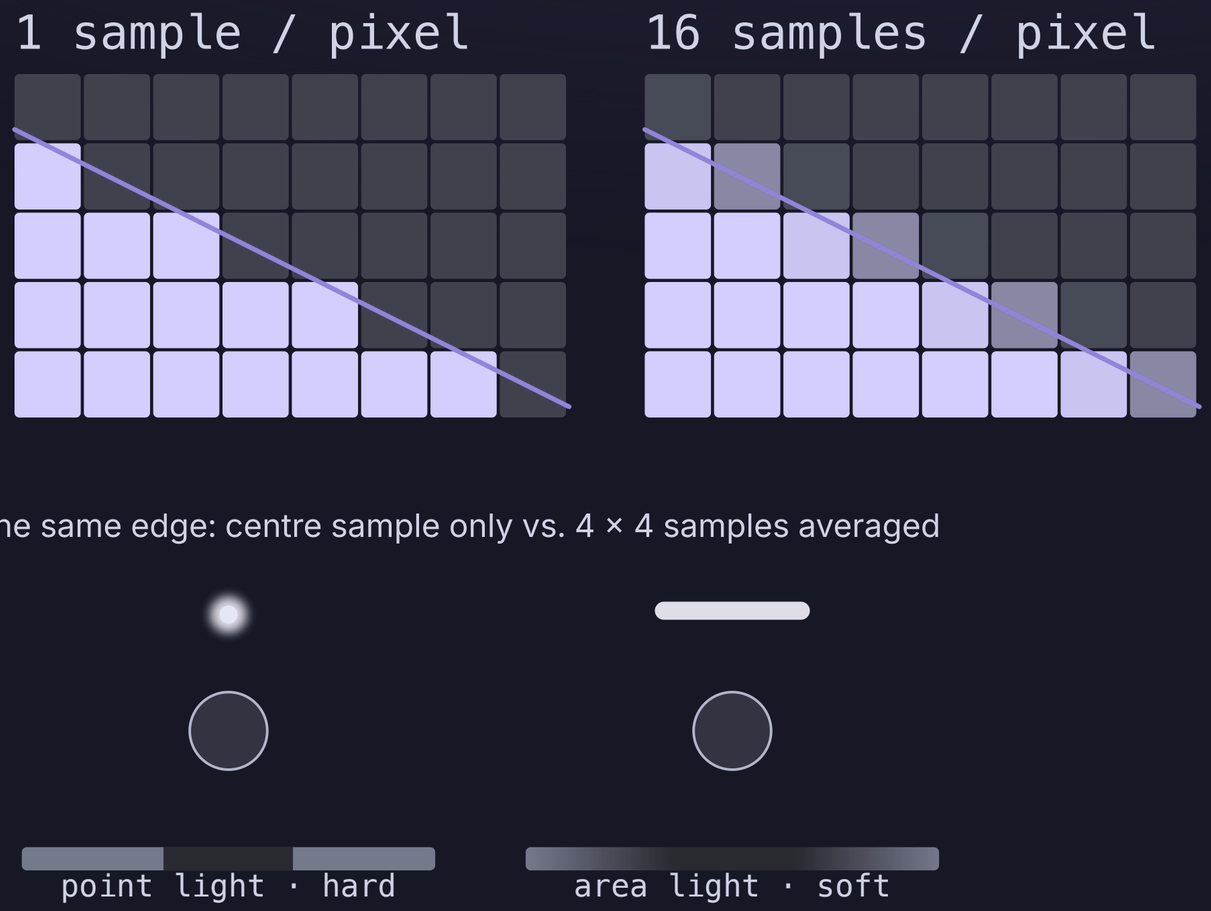
- One ray per pixel is all-or-nothing: edges become *jaggies*.
- Shoot n rays per pixel and average: the pixel takes the fraction covered.

AREA LIGHTS

- Point light: a point is blocked or not — a *hard* shadow.
- Area light: shadow rays to many points on the light; the visible fraction gives a *soft* penumbra.

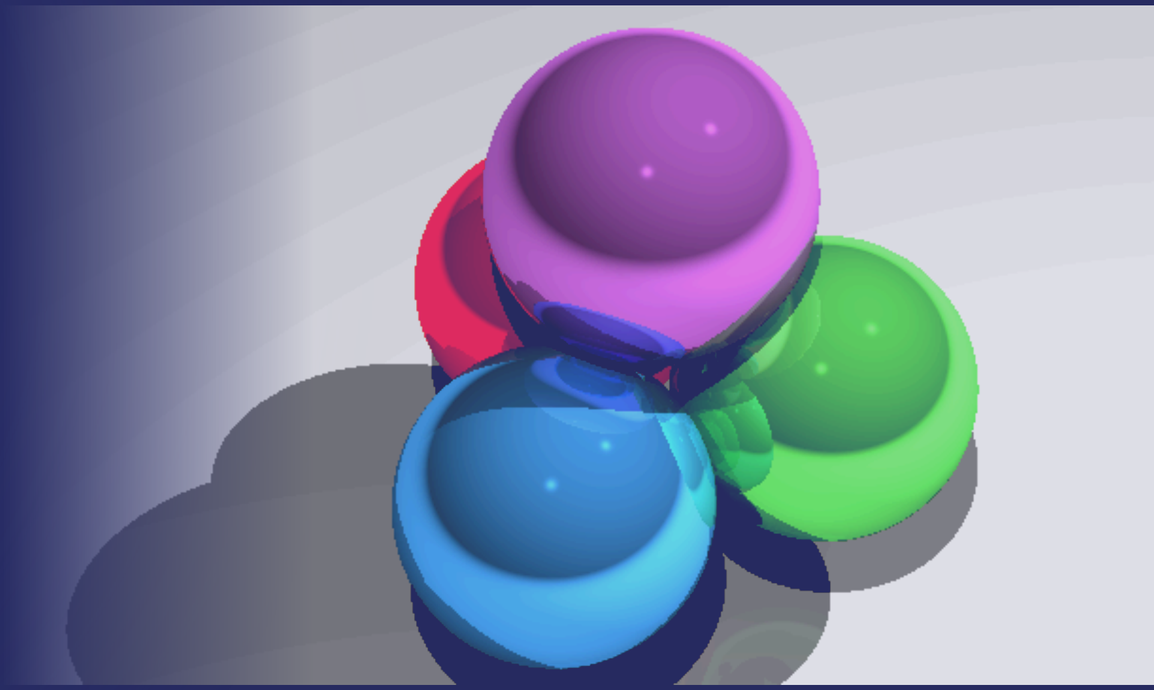
COST

- Every sample is another ray — n samples cost n×.



04

Global illumination



Direct vs indirect light

DIRECT

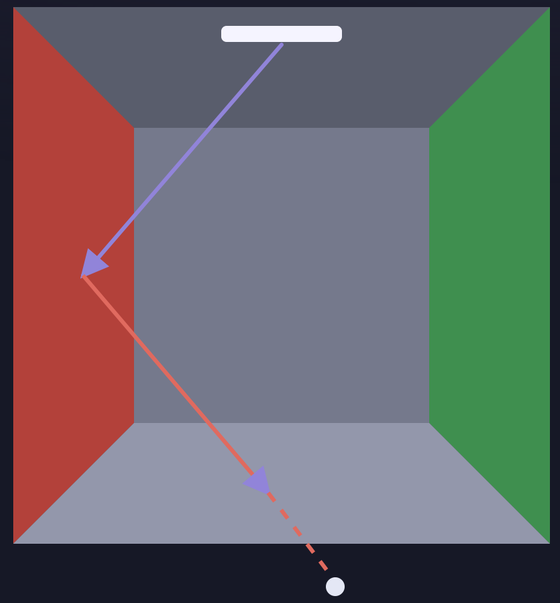
- Light → surface → eye. The only light Phong computes.

INDIRECT

- Light bounces off other things first — it reaches spots facing away from the light.
- *Colour bleeding*: light off the red wall tints the white floor red.
- Phong fakes all of it with one constant: ambient.

THE CORNELL BOX

- The standard test scene: a white room, a red and a green wall, a light in the ceiling.



light → red wall → floor → eye: the floor picks up red

Local vs global illumination

Local — Phong, Blinn

- Light bounces *once* off a surface to the eye.
- Everything else is one constant: ambient.

Global

- Follow light as it bounces between objects.
- Ray tracing does mirror bounces, but never matte to matte.
- So no colour bleeding — the red wall never tints the floor.

Light paths — E eye · L light · S shiny · D matte

`E – S* – D – L`	ray tracing — mirrors, then one matte hit
`E – D – D – D – L`	radiosity — matte to matte (colour bleeding)
`E – D – S – L`	caustics — glass focuses light onto a floor

One equation for all of it

$$\boxed{L_o(p, \omega_o)} = L_e(p, \omega_o) + \int_\Omega f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \; \mathrm{d}\omega_i$$

$L_o(p,\omega_o)$	light leaving p toward the eye

One equation for all of it

$$L_o(p, \omega_o) = \boxed{L_e(p, \omega_o)} + \int_\Omega f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \; \mathrm{d}\omega_i$$

$L_o(p,\omega_o)$	light leaving p toward the eye
$L_e(p,\omega_o)$	light the surface gives off itself (a lamp)

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_\Omega f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \, \mathrm{d}\omega_i$$

$L_o(p,\omega_o)$	light leaving p toward the eye
$L_e(p,\omega_o)$	light the surface gives off itself (a lamp)
$\int_\Omega \dots \, \mathrm{d}\omega_i$	add up over every direction light can arrive from

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_\Omega f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot (n \cdot \omega_i) \; \mathrm{d}\omega_i$$

$L_o(p,\omega_o)$	light leaving p toward the eye
$L_e(p,\omega_o)$	light the surface gives off itself (a lamp)
$\int_\Omega \dots \mathrm{d}\omega_i$	add up over every direction light can arrive from
$f_r(p,\omega_i,\omega_o)$	the material — how much arriving light it sends toward the eye

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_\Omega f_r(p, \omega_i, \omega_o) \cdot \boxed{L_i(p, \omega_i)} \cdot (n \cdot \omega_i) \; \mathrm{d}\omega_i$$

$L_o(p,\omega_o)$	light leaving p toward the eye
$L_e(p,\omega_o)$	light the surface gives off itself (a lamp)
$\int_\Omega \dots \; \mathrm{d}\omega_i$	add up over every direction light can arrive from
$f_r(p,\omega_i,\omega_o)$	the material — how much arriving light it sends toward the eye
$L_i(p,\omega_i)$	light arriving from that direction — another surface's outgoing light

One equation for all of it

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_\Omega f_r(p, \omega_i, \omega_o) \cdot L_i(p, \omega_i) \cdot \boxed{(n \cdot \omega_i)} \, \mathrm{d}\omega_i$$

$L_o(p,\omega_o)$	light leaving p toward the eye
$L_e(p,\omega_o)$	light the surface gives off itself (a lamp)
$\int_\Omega \dots \, \mathrm{d}\omega_i$	add up over every direction light can arrive from
$f_r(p,\omega_i,\omega_o)$	the material — how much arriving light it sends toward the eye
$L_i(p,\omega_i)$	light arriving from that direction — another surface's outgoing light
$(n \cdot \omega_i)$	the tilt factor — slanted light spreads thinner

Solving it, and ray tracing's limits

$$L_o(x,\omega) = L_e(x,\omega) + \int_\Omega f_r(x,\omega',\omega) \cdot L_i(x,\omega') \cdot (\omega' \cdot n) \quad \mathrm{d}\omega'$$

light out = light given off + (light arriving × material × tilt), over every direction. Phong only looks at the lights and fakes the rest.

Solving it

- Monte Carlo: estimate the sum with random samples.
- Area lights: soft shadows instead of hard ones.

Limits of ray tracing

- Slow → bounding volumes.
- Rays multiply each bounce → random sampling.
- Misses matte-to-matte bouncing.

Faster intersections

- Wrap objects in simple shapes; skip what clearly misses.
- Split space into nested boxes.
- Project boxes onto the screen to skip whole groups of rays.

What to bring into the test

PIXEL ALGORITHMS
Loop over pixels. For recursion, mark visited *before* recursing.

PIXEL MAPPING
Loop over the output and pull. Last column is W − 1 — check one corner.

COLOUR
Share bits by the brightness weights — green gets most. Ink = 1 − light.

COMPOSITING
Alpha is coverage. Top layer takes its share, the bottom shows through.

GAMMA
$I = a^{\gamma}$: take logs to solve. 0 and 1 never move.

RAYS
Ray = eye + t·direction. +0.5 for the pixel centre.

CAMERAS
Perspective: one start, many directions. Orthographic: one direction, many starts.

NEW SHAPES
Intersect each piece, keep hits on the part that exists, sort.

BOUNDING VOLUMES
Min/max for a box. Boxes suit long shapes, spheres round ones.

SHADING
Diffuse needs l; specular also needs the eye. Bigger p = tighter.

SHADOWS
Ray to the light: only hits with $0 < t < 1$ block it.

REFRACTION
Into glass bends toward the normal; out, away.

Questions?

Reading: Fundamentals of Computer Graphics, Ch. 3–4 ·
csc317tas@cs.toronto.edu

